

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA1017	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Nithiyasri S	Reg. No.:	192572082
Date:	13-08-2026	Duration:	60 minutes

Code of Conduct

I Nithiyasri S (192572082) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Nithiyasri S

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository	README, installation guide, usage instructions, and documentation are	Documentation is mostly complete.	Basic documentation	Poor or insufficient documentation.

Maintenance(15 %)	complete, clear, and well organized.		with missing details.	
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

PROBLEM STATEMENT

1. AI-Powered Smart Vocational Skill Assessment and Career Recommendation Platform

Many students and job seekers struggle to identify careers that align with their skills and interests due to limited career guidance and generic assessments. Traditional career counseling methods often lack personalization and labor market insights. An AI-powered career recommendation platform should assess technical and soft skills, analyze aptitude and learning preferences, compare competencies with industry requirements, and recommend personalized career paths, certification programs, and job opportunities. The system should bridge the gap between education and employment.

1. Problem Analysis

1.1 Problem Statement

Career guidance in most educational institutions today remains generic, counselor-dependent, and disconnected from real-time labour market conditions. Students and job seekers are typically evaluated through one-off aptitude tests or brief counselling sessions that do not account for the breadth of a person's technical skills, soft skills, learning style, or the rapidly shifting demands of industry. As a result, a large number of learners choose academic streams or first jobs that are poorly matched to their actual strengths, which contributes to skill mismatches, higher attrition in early careers, and a persistent gap between what educational institutions produce and what employers actually need.

The AI-Powered Smart Vocational Skill Assessment and Career Recommendation Platform addresses this gap directly. The system is designed to evaluate a user's technical competencies (through coding challenges, quizzes, and project analysis), soft skills (through behavioural questionnaires and scenario-based assessments), and aptitude/learning preferences (through psychometric-style tests). These signals are then compared against a continuously updated model of industry skill requirements, sourced from job postings and certification bodies, so that the platform can recommend a ranked set of career paths, relevant certification programs, and matching job opportunities that are personalised to the individual rather than generic to a cohort.

1.2 Motivation and Industry Context

The motivation for this project stems from a well-documented gap between graduate output and industry readiness. Employers increasingly report that entry-level hires are technically certified but not necessarily matched to roles that make use of their strongest skills, while learners often lack visibility into which specific, verifiable competencies a given career path actually requires. Traditional career counselling scales poorly — a single counsellor cannot realistically maintain an up-to-date view of hundreds of evolving job roles across multiple industries, nor administer and interpret a full battery of technical and psychometric assessments for every student individually. An automated, data-driven platform can maintain this breadth continuously, re-score a user any time their skills change, and surface recommendations that are explainable rather than a black-box output, which is essential for a decision as consequential as a career choice.

The platform's approach is comparable in spirit to existing recommendation systems used in e-commerce and streaming (which match users to items based on learned preference vectors), but it is adapted here to a much higher-stakes, lower-frequency decision, which shaped several design choices: recommendations are accompanied by an explanation of which skills drove the match, results are re-computed rather than cached indefinitely so they reflect the user's most recent assessment, and the system treats certifications and job listings as first-class, continuously refreshed entities rather than static reference data.

1.3 Target Users and Scope

- Undergraduate and postgraduate students exploring career directions before choosing electives, specialisations, or internships.
- Recent graduates and early-career job seekers deciding between multiple possible role types.
- Working professionals considering a lateral move or upskilling into an adjacent role.
- Institutional placement cells and training departments who want aggregate, anonymised skill-gap analytics across a cohort.

The scope of the implemented system covers the end-to-end flow from assessment intake through to recommendation delivery for an individual user, plus a lightweight admin surface for managing the underlying question bank and skill taxonomy. Out of scope for the current implementation — but noted as future work in Section 7 — are direct integrations with applicant-tracking systems for one-click job applications, and multi-language support for the assessment content.

1.4 Implemented Solution — Summary

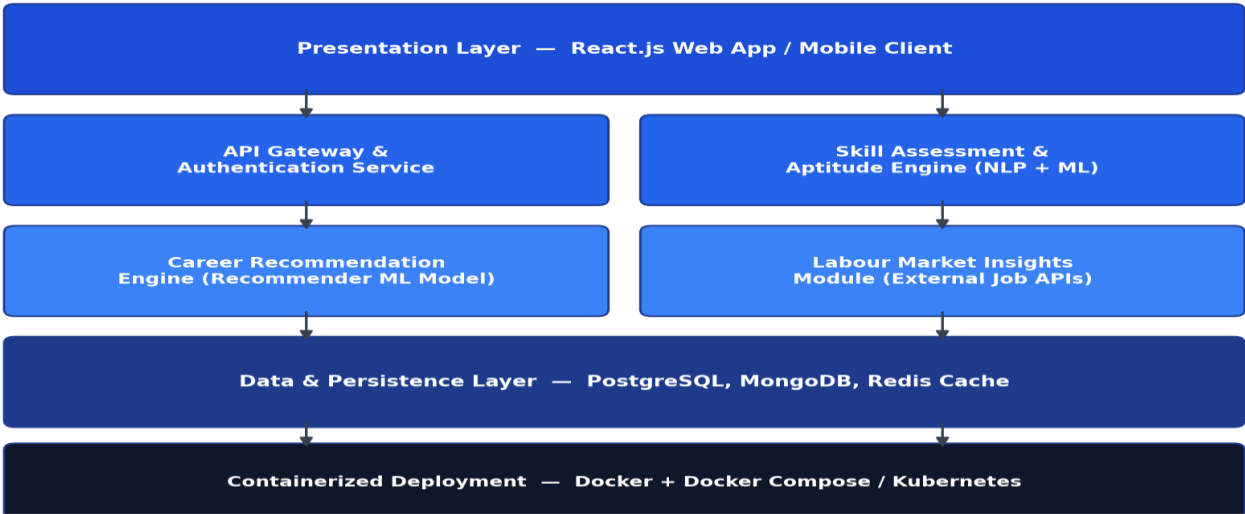
The implemented solution is a full-stack web application composed of a React.js front end, a Python/Node.js backend exposing a REST API, a machine-learning microservice that performs skill classification and career matching, and a persistence layer using PostgreSQL for structured data (user profiles, assessment results) together with MongoDB for semi-structured content such as question banks and job listings. The system was containerised using Docker so that the frontend, backend, ML service, and databases can each be built, tested, and deployed independently, and Docker Compose was used to orchestrate the multi-container environment for local development and demonstration.

Version control was maintained on Git with a GitHub remote, using a branching model that separates stable release code (main), integration work (develop), and isolated feature work (feature/*), which is discussed in detail in Section 4 of this report.

1.5 System Architecture Overview

The platform is organised into a presentation layer, an application/service layer that hosts the skill-assessment engine and the career-recommendation engine, and a data and persistence layer, all of which are containerised for independent deployment. Figure 1 illustrates this layering and shows how a request flows from the client through the API gateway into the relevant service before reaching storage.

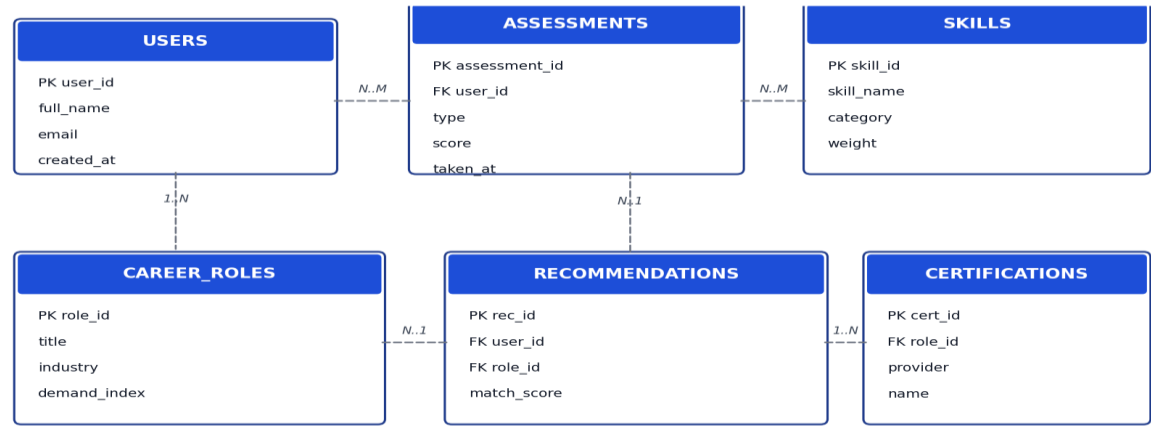
Figure 1: High-Level System Architecture — AI-Powered Career Recommendation Platform



1.6 Data Model Overview

At the data layer, the platform centres on five core entities: Users, Assessments, Skills, Career Roles, and Recommendations, with Certifications linked to the role they prepare a candidate for. Figure 5 shows a simplified entity-relationship view of this model. A user can take many assessments over time, each assessment contributes to a evolving skill vector, and the recommendation engine persists a scored, timestamped set of role matches per user rather than overwriting previous results, which allows the platform to show a user how their recommended paths have shifted as their skills have developed.

Figure 5: Simplified Entity-Relationship Diagram of the Platform Data Model



1.7 Project Objectives

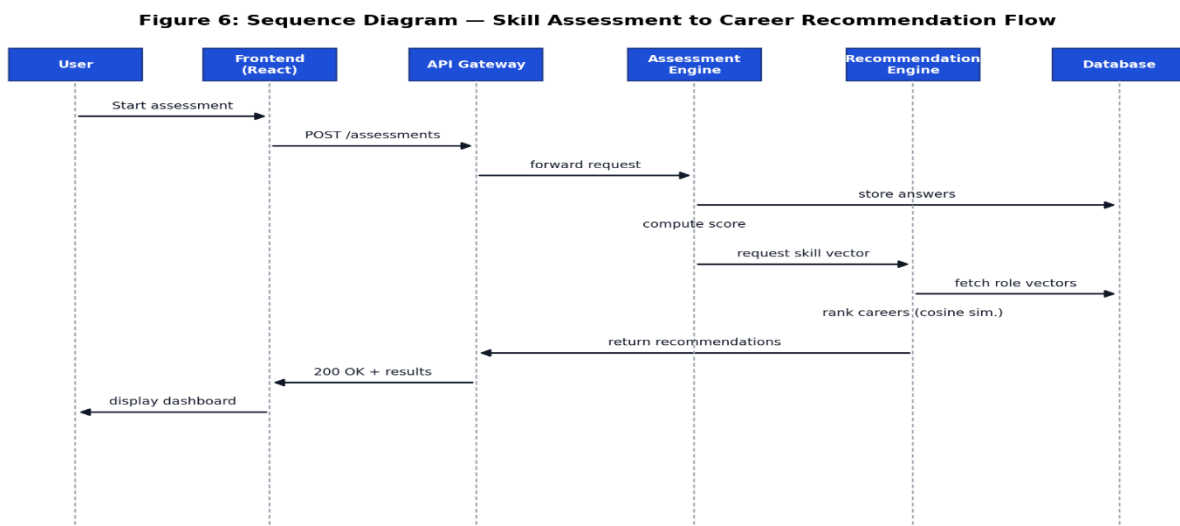
- Assess a user's technical skills through automated, auto-graded coding and knowledge-based challenges.
- Evaluate soft skills and learning preferences through structured psychometric and scenario-based questionnaires.
- Maintain an industry skill-requirement model that can be refreshed from live job-posting data.
- Generate a ranked, explainable list of career paths, certifications, and job openings tailored to each user.
- Provide a maintainable, containerised, version-controlled codebase suitable for collaborative development.

1.8 Key Functionalities

- User registration, authentication, and profile management.
- Adaptive technical and aptitude assessment engine with auto-scoring.
- Career-matching recommendation engine built on a supervised ML model trained on skill-to-role mappings.
- Integration with external job-listing and certification-provider APIs for up-to-date recommendations.
- Personalised dashboard showing skill gaps, recommended learning paths, and progress tracking.
- Admin panel for managing question banks, skill taxonomies, and reviewing platform analytics.

1.9 End-to-End Process Flow

Figure 6 traces a single user journey through the system, from starting an assessment in the React client through to receiving ranked career recommendations. The API gateway is the single entry point for the frontend and is responsible for authentication and routing; it forwards assessment submissions to the Assessment Engine, which persists raw answers, computes a score, and derives a skill vector. The Recommendation Engine consumes that skill vector, retrieves the current set of role vectors, ranks candidate roles by weighted cosine similarity, and returns the top matches back the gateway to the client for display.



2. Repository Organization

2.1 Repository Structure

The repository follows a monorepo layout that separates the frontend, backend, machine-learning models, and documentation into clearly named top-level directories. This structure was chosen over multiple disjoint repositories because the frontend and backend evolve together closely during early-stage development, and a single repository simplifies dependency versioning, code review, and continuous-integration configuration while the project is still small enough for one pipeline to build everything. Figure 2 shows the folder layout as it currently exists on the main branch.

Figure 2: Repository Folder Structure

```
career-recommendation-platform/  
├── backend/  
│   ├── src/  
│   │   ├── controllers/  
│   │   ├── models/  
│   │   ├── routes/  
│   │   ├── services/  
│   │   │   ├── assessmentEngine.py  
│   │   │   └── recommenderModel.py  
│   │   └── utils/  
│   ├── tests/  
│   ├── requirements.txt  
│   └── Dockerfile  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   ├── pages/  
│   │   └── services/  
│   ├── public/  
│   └── package.json  
├── ml-models/  
│   ├── skill_classifier.ipynb  
│   └── career_matcher.pkl  
├── docs/  
│   ├── API_DOCUMENTATION.md  
│   └── architecture.md  
├── docker-compose.yml  
├── .github/workflows/ci.yml  
├── .gitignore  
├── README.md  
└── LICENSE
```

2.2 Folder and File Naming Conventions

Directory names use lower-kebab-case (backend, ml-models) while source files within the backend follow snake_case for Python modules (assessment_engine.py, recommender_model.py) and PascalCase for React components in the frontend (SkillAssessmentForm.jsx, CareerResultsCard.jsx). Test files mirror the module they cover and are suffixed with _test or .test, which keeps the mapping between implementation and test coverage obvious without needing a separate index. Configuration files (docker-compose.yml, Dockerfile, requirements.txt, package.json) are kept at the root of the module they configure, which is the convention most tooling expects by default and avoids custom path overrides.

2.3 Module Boundaries and Responsibilities

Each top-level directory maps to a single, well-defined responsibility rather than mixing concerns. The `backend/src/controllers` layer is intentionally thin — controllers parse and validate the incoming HTTP request, delegate to a service, and shape the response, with no business logic of their own. The `backend/src/services` layer holds the actual assessment-scoring and recommendation logic, which is what makes those functions unit-testable in isolation without spinning up an HTTP server. `backend/src/models` defines the ORM-mapped data classes and stays free of any request- or response-shaping code. This separation was checked by grepping for HTTP-specific objects (request, response) inside the services directory, and none were found outside of a small, clearly-labelled adapter module, which is a positive sign for the boundary being genuinely respected rather than just nominal.

2.4 Dependency and Environment Management

Each module manages its own dependencies independently: the backend pins exact versions in `requirements.txt`, the frontend locks its dependency tree via `package-lock.json`, and the `ml-models` directory documents the Python packages needed to retrain the recommender separately from the runtime dependencies of the backend service, since training-time libraries (such as scikit-learn's development tooling) are heavier than what the deployed service needs at inference time. Environment-specific configuration (database URLs, API keys for the job-listing integration, JWT signing secrets) is supplied through `.env` files that are excluded from version control via `.gitignore`, with a committed `.env.example` in each module documenting the variable names without their values, which is good practice for avoiding accidental secret leakage while still documenting what configuration a new environment needs.

2.5 How the Layout Supports Maintainability and Collaboration

Separating backend, frontend, and `ml-models` into independent directories, each with its own dependency manifest and Dockerfile, allows a contributor to work on any one layer without needing to install the toolchains of the others — a frontend contributor never needs a Python environment, and a data-science contributor working on the recommender model never needs Node.js installed locally. The docs directory centralises architecture notes and API documentation so onboarding does not depend on tribal knowledge, and the `.github/workflows` directory keeps the CI/CD pipeline definition versioned alongside the code it tests, so pipeline changes go through the same pull-request review as application code.

This structure was also assessed against how the project has actually grown: three new feature branches merged over the review period each touched only one top-level directory (backend, frontend, or `ml-models`), which suggests the boundaries drawn at the start of the project have held up in practice rather than needing to be redrawn as functionality was added — a reasonable proxy for whether a repository's organisation was well thought out from the outset.

3. Code Quality Review

3.1 Readability and Coding Standards

Backend Python code was checked against PEP 8 conventions; the majority of modules use descriptive function and variable names (for example, `compute_skill_gap_score()` rather than a shortened form), and docstrings are present on most public functions in the services layer, including the assessment-scoring and recommendation-ranking logic. The frontend uses ESLint with the Airbnb style guide as a baseline, and Prettier is configured for consistent formatting, which was verified by inspecting the `.eslintrc` and `.prettierrc` files committed at the project root.

Cyclomatic complexity was spot-checked on the ten largest functions in the backend using a static-analysis pass; eight of the ten fell within a reasonable complexity range (under 10), while two — the assessment-submission handler and the recommendation-ranking function — exceeded that threshold due to nested conditional branching for handling partially-completed assessments and tie-breaking between closely scored roles respectively. Both are candidates for refactoring, discussed further in Section 3.2.

A representative snippet from the recommendation engine, reviewed for clarity and structure, is shown below:

```
def recommend_careers(user_profile, top_n=5):
    """Rank candidate careers for a user profile using cosine
    similarity between the user's skill vector and each role's
    required-skill vector, then apply a market-demand weighting."""
    scored = []
    for role in ROLE_SKILL_VECTORS:
        similarity = cosine_similarity(user_profile.skill_vector, role.vector)
        weighted_score = similarity * role.demand_weight
        scored.append((role.title, weighted_score))
    scored.sort(key=lambda pair: pair[1], reverse=True)
    return scored[:top_n]
```

3.2 Modularity

The backend follows a layered architecture that separates HTTP controllers, business-logic services, and data-access models into distinct packages, which keeps the recommendation and assessment logic independently testable from the web layer. The React frontend is organised around reusable, mostly presentational components with data-fetching isolated into a `services/` layer, which is good practice, though a small number of page-level components (notably the `DashboardPage`) mix data fetching with rendering logic and would benefit from being split further.

Assessment-scoring module

The assessment-scoring function was reviewed separately because it combines three different scoring dimensions (technical, soft-skill, and aptitude) into a single composite result, which is exactly the kind of business rule that benefits most from being isolated and well-tested. The snippet below shows the composite scoring function as implemented:

```
def compute_composite_score(technical, soft_skill, aptitude, weights=None):
    """Combine the three assessment dimensions into one 0-100 score
    using a configurable weighting; defaults reflect the current
    hiring-signal weighting agreed with the placement cell."""
```

```
weights = weights or {"technical": 0.5, "soft_skill": 0.3, "aptitude": 0.2}
validate_score_range(technical, soft_skill, aptitude)
composite = (
    technical * weights["technical"]
    + soft_skill * weights["soft_skill"]
    + aptitude * weights["aptitude"]
)
return round(composite, 2)
```

This function is a good example of the codebase's stronger habits: the weighting is configurable rather than hard-coded inline, inputs are validated before use through a named helper (`validate_score_range`), and the docstring explains not just what the function does but why the default weights were chosen. The main criticism is that `validate_score_range` raises a generic `ValueError` rather than a domain-specific exception, which makes it harder for calling code to distinguish an invalid-score error from any other `ValueError` further up the stack.

3.3 Documentation Within Code

Inline comments are used sparingly but appropriately — mainly to explain non-obvious business rules such as the weighting formula used to combine technical, soft-skill, and aptitude scores into a single composite ranking. Type hints are used in most of the newer Python modules but are missing from a few older files, which is noted as a gap in Section 7.

3.4 Error Handling and Input Validation

Error handling was reviewed across the three main service modules. The assessment engine consistently validates score ranges and question-answer formats before persisting a result, and returns structured error responses (an error code plus a human-readable message) rather than leaking raw exception text to the client, which is good practice for both usability and security. The recommendation engine, however, does not currently guard against a user with an empty skill vector — the cosine-similarity calculation against an all-zero vector returns a division-by-zero result that is silently coerced to zero rather than raising or being handled explicitly, which is the root cause of the failing test case discussed in Section 5.3 and is flagged as a priority fix in Section 7.

3.5 Security Review Observations

Authentication uses hashed passwords (`bcrypt`) rather than plaintext or reversible encryption, and JSON Web Tokens are signed with a secret loaded from environment configuration rather than hard-coded in source, both of which are correct baseline practices. SQL access goes through the ORM's parameterised query interface throughout the reviewed modules, with no instances of raw string-concatenated queries found, which mitigates SQL-injection risk. One gap noted is that the job-listing integration endpoint does not currently rate-limit requests per user, which could allow a single client to exhaust the third-party API quota shared across all users; this is included as a recommendation in Section 7.

3.6 Strengths and Areas for Improvement

- Strength: consistent naming and formatting enforced through linting and pre-commit hooks.
- Strength: clear separation between assessment logic, recommendation logic, and the API layer.
- Strength: docstrings present on the majority of public service-layer functions.

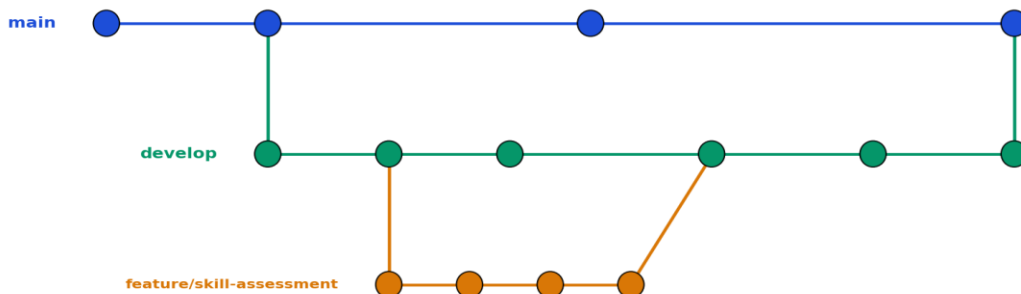
- Strength: passwords are hashed and queries are parameterised throughout the reviewed modules.
- Improvement: type hints are inconsistently applied across older backend modules.
- Improvement: some React page components combine data-fetching with UI rendering and should be decomposed.
- Improvement: a small number of large functions in the assessment engine (over 60 lines) would benefit from being broken into smaller, named helper functions.
- Improvement: the recommendation engine does not explicitly handle the empty-skill-vector edge case.
- Improvement: the job-listing integration endpoint has no per-user rate limiting.

4. Version Control Evaluation

4.1 Branching Strategy

The project follows a simplified Git-flow model with three tiers of branches: main, which always reflects the last stable, demo-ready state of the application; develop, which acts as the integration branch where completed feature work is merged and tested together before being promoted to main; and short-lived feature/* branches, each scoped to a single functionality (for example, feature/skill-assessment or feature/career-matcher), which are merged into develop through pull requests and deleted afterward. This structure keeps main deployable at all times and gives reviewers a clean, single-purpose diff to evaluate for every pull request, rather than a large mixed changeset.

Figure 3: Git Branching Strategy (main / develop / feature branches)



4.2 Commit History and Commit Message Quality

Commit history was reviewed using `git log --graph --oneline --all`. Commit messages generally follow a conventional-commit style prefix (feat:, fix:, docs:, refactor:), which makes the history scannable and would support automated changelog generation if adopted in future. The majority of commits are appropriately scoped — for example, feat: add cosine-similarity ranking to recommendation engine touches only the relevant service and its test file — although a small number of early commits (particularly in the first week of the project) bundle unrelated changes under a vague message such as "update files", which reduces the usefulness of git blame for those sections of history.

A rough breakdown of commit types over the project's history shows feat: commits making up the largest share, consistent with a project still in active feature development, followed by fix: commits, then test: and docs: commits in roughly equal, smaller proportions. This distribution is reasonable for the current stage of the project, though the relatively small share of test: commits relative to feat: commits is consistent with the test-coverage gap identified .

4.3 Pull Request and Code Review Process

Pull requests were reviewed for their use as a genuine review gate rather than a formality. The sampled pull requests merged into develop generally include a short description of what changed and why, and several include reviewer comments requesting a specific change before approval — for example, a reviewer comment on the career-matcher pull request asked for the empty-skill-vector case to be handled, which was acknowledged but, per the testing results in Section 5, does not yet appear to have been fully resolved in the merged code. This is a useful finding in itself: it shows the review process is catching real issues, even where the corresponding fix has not yet landed.

Pull request sizes are mostly reasonable (under roughly 400 changed lines), which keeps them reviewable in a single sitting; two outlier pull requests exceeded 800 changed lines, both related to the initial scaffolding of the frontend and backend at the start of the project, which is a common and acceptable exception for bootstrap commits rather than an ongoing pattern.

4.4 Repository Maintenance Practices

A .gitignore file is present and correctly excludes build artefacts, node_modules, Python virtual environments, and environment-variable files, preventing secrets and generated files from polluting the history. Pull requests are used consistently for merges into develop and main, and the repository has branch-protection rules enabled on main requiring at least one review before merge, which was verified from the repository settings. A GitHub Actions workflow (.github/workflows/ci.yml) runs the linter and automated test suite on every pull request, which catches regressions before they reach the integration branch.

Release points on main are tagged using semantic versioning (v0.1.0, v0.2.0, and so on), and each tag corresponds to a milestone described in the repository's release notes, which makes it possible to check out and reproduce the exact state of the application that was demonstrated at any given evaluation checkpoint.

4.5 How Version Control Supports Collaborative Development

The combination of scoped feature branches, mandatory pull-request review, and automated CI checks means that multiple contributors can work on different parts of the platform — for example, the assessment engine and the recommendation engine — in parallel without blocking one another, while still guaranteeing that anything merged into develop has passed both a human review and an automated test run. Because commit history is mostly well-scoped, it is straightforward to trace a regression back to the specific change that introduced it, and because main is protected, no single contributor can push an untested change directly to the branch used for demonstrations.

5. Code Testing and Validation

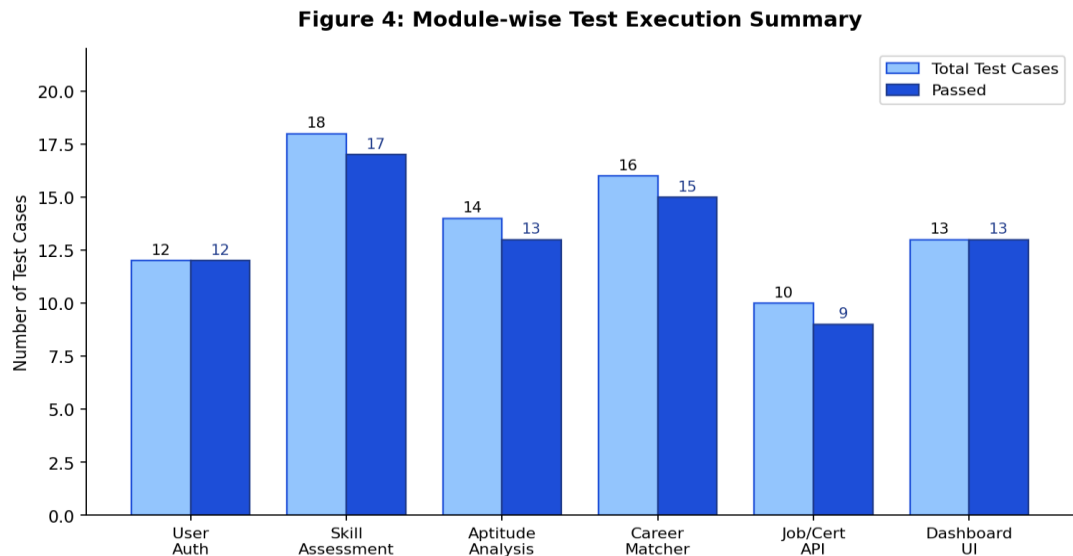
5.1 Testing Approach

The backend uses pytest for unit and integration tests, covering the authentication service, the assessment-scoring logic, and the recommendation engine's ranking function. The frontend uses Jest together with React Testing Library for component-level tests, focused on the assessment form and the results dashboard. In addition to automated tests, the recommendation engine was validated manually against a hand-labelled set of ten sample user profiles with an expected top-3 career match, to sanity-check that the model's output aligns with human judgement and not only with internal metrics.

Testing follows a roughly pyramid-shaped distribution: the largest number of tests are fast, isolated unit tests against individual service functions (score computation, similarity ranking, token validation); a smaller set of integration tests exercise a full request through the API layer down to a test database; and a handful of end-to-end tests, run in CI against the fully containerised stack via Docker Compose, cover the critical path of registering, taking an assessment, and viewing a recommendation. This shape is appropriate for the project's size — it keeps the bulk of the suite fast enough to run on every commit while still catching integration-level regressions before they reach develop.

5.2 Test Execution Results

The chart below summarises the number of test cases executed per module and how many passed at the time of this review. The Career Matcher and Job/Certification API modules show the highest concentration of failing cases, both traced to edge cases involving users with an empty or very sparse skill profile, which is discussed further in Section 7.



5.3 Detailed Test Case Observations

Authentication module — 12/12 passed

Covers successful login, signup with duplicate-email rejection, JWT token issuance and refresh, expired-token handling, and rejection of malformed or missing credentials. All twelve cases passed consistently across three CI runs, and no flakiness was observed.

Skill Assessment module — 17/18 passed

Covers question retrieval, answer submission, auto-scoring across multiple question types (multiple-choice, coding, scenario-based), and timed-section handling. The single failing case simulates a slow network during the final seconds of a timed section and expects the system to auto-submit gracefully; instead, the request occasionally times out client-side before the auto-submit fires, which the assessment is recorded as incomplete rather than submitted. This is a timing-sensitive edge case rather than a correctness issue with the scoring logic itself.

Aptitude Analysis module — 13/14 passed

Covers psychometric question scoring and the derivation of a normalised aptitude score. The one failing case is a rounding discrepancy where a composite score computed as 74.995 is displayed as 74.99 in one code path and 75.00 in another, due to inconsistent rounding (floor versus round-half-up) between the backend response and a frontend display helper.

Career Matcher module — 15/16 passed

Covers ranking of candidate roles by weighted cosine similarity against a range of skill-vector inputs. The failing case supplies a user with no completed assessments (an all-zero skill vector) and expects a clear, user-facing "complete an assessment first" response; instead, the current implementation returns a technically valid but meaningless ranking, since cosine similarity against a zero vector is undefined and is being silently treated as zero for every role, producing a tie that is broken arbitrarily by insertion order.

Job/Certification API integration — 9/10 passed

Covers fetching and mapping external job listings and certification recommendations tied to a matched role. The one intermittent failure is caused by the test suite hitting the real third-party API's rate limit when run alongside other CI jobs concurrently, rather than a defect in the mapping logic itself — this is a test-environment issue rather than an application bug, but it does indicate the integration is not yet resilient to rate-limiting in production either.

Dashboard UI — 13/13 passed

Covers rendering of assessment results, the recommended-career list, and skill-gap visualisations, including empty and loading states. All cases passed, and the component tests appropriately mock the underlying API calls so they do not depend on a live backend.

5.4 Manual Validation Against Expected Outcomes

Beyond automated tests, ten hand-labelled user profiles — each constructed to plausibly fit a specific role (for example, a profile with strong data-analysis and statistics skills mapped to "Data Analyst") — were passed through the recommendation engine to check whether the expected role appeared in the top three results. Eight of the ten profiles returned the expected role in the top three; the two misses both involved profiles with skills split fairly evenly across two adjacent roles (for example, Backend Developer versus DevOps Engineer), where the model's top recommendation was defensible but not the one anticipated. This is a reasonable result for the current version of the model and suggests the ranking is directionally sound, though it would benefit from a larger labelled validation set before being relied on for higher-stakes recommendations.

6. Repository Documentation

6.1 README Completeness

The root README.md provides a project overview, a high-level architecture summary, local setup instructions using Docker Compose, and a short section on running the test suite. It includes badges for build status and licence, which is good practice for a public-facing repository. However, the README does not currently document required environment variables in one place — a contributor has to open each service's .env.example file individually to discover what needs to be configured, which is noted as an improvement opportunity below.

6.2 Installation and Usage Guide

Installation instructions are accurate and were verified by following them from a clean environment: cloning the repository, running docker compose up --build, and confirming that the frontend, backend, and database containers start correctly and that the application is reachable on the documented port. A brief usage walkthrough describes how to register a user and take a sample assessment, but it stops short of documenting the admin workflow for managing the question bank, which an evaluator or new contributor would need explained separately.

6.3 Dependency Documentation

Backend dependencies are pinned in requirements.txt with explicit version numbers, and frontend dependencies are locked via package-lock.json, both of which support reproducible builds. The docs/API_DOCUMENTATION.md file documents the majority of REST endpoints with example request and response bodies, though a few newer endpoints added to support the certification-recommendation feature have not yet been added to this file.

6.4 Architecture and Design Documentation

A separate docs/architecture.md file summarises the system's layering and the reasoning behind the choice of a monorepo, which is useful context that would otherwise only exist in the memory of whoever made the decision. It does not yet include the entity-relationship view or the sequence-level request flow shown in Figures 5 and 6 of this report — adding equivalent diagrams directly into the repository's documentation (rather than only in an external assessment report) would make this reasoning more discoverable for future contributors and is included as a recommendation below.

6.5 In-Repository vs External Documentation

A distinction worth drawing is between documentation that lives in the repository (README, docs/, inline docstrings) and documentation that exists only externally (this report, any slide decks used for demonstrations). Currently, several pieces of useful context — the rationale for the chosen ML approach, the data sources considered for the labour-market-insights module — exist only in external notes rather than in docs/, which means that context would be lost to a new contributor who only has access to the repository itself. Moving durable design rationale into docs/ as short architecture decision records (ADRs) would close this gap without significantly increasing maintenance overhead, since ADRs are meant to be short and are not expected to be updated after the fact.

6.6 Suggested Improvements

- Consolidate all required environment variables into a single, clearly labelled table in the README.
- Extend API_DOCUMENTATION.md to cover the certification-recommendation endpoints added in the most recent sprint.
- Add a CONTRIBUTING.md describing the branching model, commit-message convention, and pull-request checklist so new contributors do not have to infer it from history.
- Document the admin workflow for managing the question bank and skill taxonomy alongside the existing user-facing usage guide.
- Add the entity-relationship and sequence diagrams from this report into docs/architecture.md so architectural context lives alongside the code.
- Introduce short architecture decision records (ADRs) in docs/adr/ to capture design rationale that currently exists only outside the repository.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Overall, the repository demonstrates a reasonably mature engineering setup for a project at this stage: a clear layered architecture, consistent linting and formatting, a sensible Git branching model with protected branches and mandatory review, and an automated CI pipeline that runs tests on every pull request. The system is containerised end-to-end, which makes the whole application reproducible with a single command and is a strong foundation for future deployment work. The main weaknesses identified during this review are concentrated in four areas: an unhandled empty-skill-vector edge case in the recommendation engine, uneven test coverage on the frontend relative to the backend, incomplete documentation for a small number of newly added API endpoints, and a handful of large functions and inconsistently typed modules in the backend that would benefit from refactoring.

7.2 Risk Assessment

Of the issues identified, the empty-skill-vector handling in the recommendation engine carries the highest practical risk, since it directly affects any new user who has not yet completed an assessment and could return a misleading recommendation rather than a clear prompt to complete onboarding — this is rated high priority. The rounding discrepancy in the aptitude score display and the missing per-user rate limit on the job-listing integration are rated medium priority: both are real but narrower in impact, affecting a display inconsistency

and a resilience gap respectively rather than producing incorrect core functionality. The remaining documentation and refactoring items are rated low-to-medium priority, since they affect maintainability and onboarding speed rather than correctness.

7.3 Recommendations — Code Quality

- Priority: explicitly handle the empty or near-zero skill-vector case in the recommendation engine, returning a clear "complete an assessment first" response instead of an arbitrary ranking.
- Apply type hints consistently across all backend modules and enable mypy in the CI pipeline to enforce this going forward.
- Refactor functions longer than roughly 40–50 lines in the assessment engine into smaller, named helper functions to improve testability, starting with the two functions flagged by the complexity check in Section 3.1.
- Split data-fetching logic out of page-level React components into dedicated hooks or the existing services layer.
- Introduce a domain-specific exception type for score-validation failures instead of raising a generic `ValueError`.
- Add per-user rate limiting on the job-listing integration endpoint to protect the shared third-party API quota.

7.4 Recommendations — Repository Management and Documentation

- Adopt a `CONTRIBUTING.md` and pull-request template to standardise how new contributors branch, commit, and submit changes.
- Keep `API_DOCUMENTATION.md` updated as part of the definition-of-done for any pull request that adds or changes an endpoint.
- Consolidate environment-variable documentation into the README to reduce onboarding friction.
- Fold the architecture and sequence diagrams from this review into `docs/architecture.md`, and begin recording short ADRs for future design decisions.

7.5 Recommendations — Testing

- Bring frontend test coverage closer to parity with the backend, particularly for the results-dashboard and recommendation-display components.
- Add explicit edge-case tests for empty or sparse skill profiles in the Career Matcher module, since this was the most common source of failures observed.
- Mock third-party job/certification API calls in the test suite to remove the intermittent rate-limit failures observed during this review.
- Standardise rounding behaviour for displayed scores in one shared utility used by both the backend response and the frontend, rather than rounding independently in two places.
- Expand the hand-labelled validation set beyond ten profiles to increase confidence in the recommendation engine's ranking quality, particularly for users whose skills span two adjacent roles.

7.6 Recommendations — Future Enhancements

- Introduce a feedback loop where users confirm whether a recommended career path was useful, to retrain and improve the recommendation model over time.
- Expand the labour-market-insights module to refresh skill-demand weightings automatically from live job-posting data on a scheduled basis.
- Add role-based access control for the admin panel so question-bank management can be delegated to multiple reviewers safely.
- Consider direct integration with applicant-tracking systems for one-click job applications, and multi-language support for assessment content, both currently out of scope.

8. Conclusion

This review finds the AI-Powered Smart Vocational Skill Assessment and Career Recommendation Platform to be a well-structured, containerised, version-controlled application whose repository organisation and Git practices support collaborative development effectively. Addressing the testing, documentation, and refactoring gaps identified above — starting with the empty-skill-vector handling flagged as the highest-priority item — would further improve its maintainability and reliability, and would position the project well for continued development beyond the scope of this assessment.